

Parallel & Distributed Computing (CS 429)

After-Midterm Study Notes — Simplified Language Version

1. Interconnection Networks for Parallel Processors

Interconnection networks let all processors in a parallel system talk to each other easily — they are like "roads" that connect processing units.

Type	Meaning
Static Interconnection Network	Fixed connections between nodes; the path doesn't change
Dynamic Interconnection Network	Connections can be rearranged on the fly

■ Memory tip: Static = Stuck in place. Dynamic = can Dance/change around.

A. Static Interconnection Network

- Unidirectional: data flows one way only ($A \rightarrow B$, not $B \rightarrow A$)
- Bidirectional: data flows both ways ($A \leftrightarrow B$)

(i) Fully / Completely Connected Network

Every node is directly linked to every other node — the highest amount of connection possible.

$$\text{Links} = n(n-1) / 2$$

Each node connects to $(n-1)$ other nodes

Example: If $n = 6 \rightarrow \text{Links} = 6 \times 5 / 2 = 15$, and each node connects to 5 others.

Feature	Detail
Connectivity	Highest possible — a direct link between any pair
Bandwidth / Latency	Best case: short delay, large data flow
Cost & Complexity	Very costly — needs the most wires
Data Transmission	Fast and direct (no stops along the way)

(ii) Limited Connection Network (Partial Network)

Each node connects to only some of the nodes (not all of them). It costs less but works less well than a fully connected network.

Feature	Fully Connected	Limited Connection
Connectivity	Every node $\leftrightarrow$ every node	Fewer links per node
Cost	High	Low
Bandwidth	High	Limited
Fault Tolerance	High	Lower
Scalability	Poor (grows too fast)	Good

Common examples: Ring, Tree, Linear Array, Mesh, Hypercube networks.

Linear Network

- Nodes connected in a single row/line, one after another.
- Weakness: if one node breaks down, every node after it stops working too (the chain breaks).

Ring Network

- Exactly like a linear network, but the last node is also connected back to the first node — forming a loop.

Three Cube (Hypercube) Network

- 3 nodes/switches connected in a cube shape to give backup paths and handle failures better.
- Used in data-center design where backup paths matter.

Two-Dimensional (Mesh) Network

- Nodes arranged in a grid (rows × columns), each node linked to its neighbors.
- Common in distributed computing because it gives organized, easy-to-predict communication.

B. Dynamic Interconnection Network

Unlike static networks, connections are NOT fixed — they change themselves as communication needs change.

★ 6 Key Aspects of Dynamic Networks

- Reconfiguration — paths can change
- Switching Mechanisms — how paths are set up
- Flexibility — adjusts to different traffic patterns
- Scalability — handles growth well
- Fault Tolerance — can reroute around failures
- Communication Efficiency — improved data flow

2. Code Optimization for Multi-Core CPUs & GPUs

Optimization = making software use hardware (cores, memory, cache) as well as possible, since raw hardware speed alone isn't enough.

Goals of Optimization:

- Improve speed & increase throughput
- Reduce memory delay & communication overhead
- Use less energy
- Balanced workload — no core sits idle while others are overloaded
- Better cache use & scalability

4 Types of Parallelism

Type	What It Means
Instruction-Level (ILP)	Multiple instructions run together inside one CPU pipeline
Thread-Level (TLP)	Multiple threads run at the same time on multi-core CPUs

Data-Level (DLP)	Same operation applied to many pieces of data at once (GPU/vector)
Task-Level	Independent tasks spread across different processing units

Optimization on Multi-Core CPUs

- Thread Parallelism: split a big task into smaller threads. Too many threads = extra overhead → fixed with thread pooling.
- Load Balancing: spreading work evenly; changing the schedule while running works better than fixing it in advance.
- Memory Optimization: improve cache locality.
 - Temporal locality — reuse recently used data
 - Spatial locality — access nearby memory locations together
 - Techniques: loop tiling, loop fusion, loop interchange
- False Sharing: multiple threads change different variables that happen to sit on the SAME cache line → this causes needless cache resets. Fix: add padding or line up the data properly.
- Synchronization: locks/semaphores/barriers are needed but slow things down — keep critical sections short; use lock-free/atomic operations when possible.

Optimization on GPUs

GPUs use huge parallelism: thousands of small threads, organized as Grid → Blocks → Warps → Threads.

Concept	Explanation
Warp	Group of 32 threads running the same instruction together
Warp Divergence	Threads in a warp take different paths → work is done one after another (slow)
Memory Coalescing	Nearby threads access nearby memory → combined into fewer memory operations
Shared Memory	Fast memory for threads within a block; watch out for bank conflicts
Occupancy	How many warps are active at once; more active warps = better at hiding delay
Data Transfer (CPU↔GPU)	A major slow point; use pinned memory + async streams to overlap tasks
Kernel Fusion	Combine several GPU kernels into one → less startup cost

3. Routing & Switching in Parallel/Distributed Systems

- Switching: HOW data physically moves from source to destination through in-between nodes.
- Routing: WHICH path/route the data takes.

Types of Switching

Type	How It Works	Trade-off
Circuit Switching	A fixed path is reserved before sending begins	Predictable, but wastes resources when traffic comes in bursts
Packet Switching	Data is split into packets, and each may take a different path	Shares resources well, but delay can vary and packets may need reordering
Message Switching	The whole message is stored and passed on at each node (not split)	No fixed path needed, but there's a long delay (store-and-forward delay)

Modern Switching Techniques

Technique	Behavior
Store-and-Forward	Waits for the ENTIRE packet before forwarding — reliable but slow
Cut-Through	Forwards as soon as the header is read — shorter delay
Wormhole	Splits packets into small pieces called "flits" and pushes them through the network in a pipeline — the fastest method, used in high-performance computing

Routing

Goals: cut delay, reduce traffic jams, handle failures well, increase throughput, and stay scalable.

Routing Type	Description	Trade-off
Deterministic	Fixed path, always the same (e.g., XY routing in mesh)	Simple and easy to predict, but can't adjust to traffic jams or failures
Adaptive	Path changes based on live network conditions	Better results and handles failure well, but needs more complex hardware
Oblivious	Path is chosen randomly / by a fixed chance, ignoring conditions	Simple, but not the best choice under heavy traffic

Deadlock in Routing

Deadlock = packets get stuck forever, each one waiting for a resource held by another — nobody can move.

Prevention techniques: virtual channels, resource ordering, deadlock-free routing algorithms.

Routing in NoC (Network-on-Chip) vs Distributed Systems

- NoC: connects cores inside one chip; limited space/power → needs good deterministic/adaptive routing plus virtual channels.
- Distributed Systems: routing over clusters/cloud; e.g. MPI handles routing decisions; cloud systems also balance load across servers.

Performance Metrics

Metric	Meaning
Latency	Time for data to travel from source → destination
Throughput	Amount of data successfully sent per unit of time

Trade-off: Deterministic routing = simple but less flexible. Adaptive routing = better performance but more complex.

4. Butterfly Network

A multi-stage interconnection network shaped like butterfly wings — it grows well, is easy to predict, and works well for high-performance computing.

5 Core Principles — ★ Remember: F-L-P-S-D

- Fixed Paths — predefined routes, avoids conflicts
- Logarithmic Distance — hops grow as $\log(n)$, keeps delay low
- Parallelism — supports many transfers happening at the same time
- Scalability — add nodes/stages without redesign
- Deterministic Routing — fixed, reliable delivery paths

Structure

- Multiple stages; each stage's nodes connect to nodes in the next stage.
- Each node links to 2 nodes in the next stage → every input can reach every output.
- Number of stages grows logarithmically with the number of inputs.
- Balanced, mirror-like structure (like butterfly wings) — makes routing simpler.
- Multiple backup paths exist → this helps handle failures better.

Routing Algorithms

Algorithm	Idea
Deterministic	Fixed route per source-destination pair
Adaptive	Adjusts based on congestion/traffic
Minimal	Shortest path (fewest hops)
Non-Minimal	Longer path to avoid busy/blocked nodes
Fault-Tolerant	Uses backup paths if a node/link breaks down

Communication Patterns

Pattern	Description
One-to-One	Direct transfer between two nodes
One-to-All	Broadcast — one node sends to everyone
All-to-One	Gathering — all nodes send to one node (e.g., collecting results)
All-to-All	Every node sends to every other node
Many-to-Many	Groups communicate with other groups
Pipeline	Data passed through a sequence of nodes, stage by stage

Advantages vs Challenges

Advantages	Challenges
Works well, low traffic jams	Complex to design & build

Scalable (add nodes easily)	High starting hardware cost
Short delay (log distance)	Difficult upkeep (many links)
Parallel processing support	Handling failures is complicated
Predictable performance	Limited flexibility (fixed paths)
Handles failures well (backup paths)	Scaling issues at very large size

Use Cases: High-Performance Computing (supercomputers), Data Centers, Scientific Simulations, Big Data Analytics, Real-Time Applications (gaming/trading), Telecommunications, Cloud Computing.

5. Z-Transformation in Distributed Systems

A math tool (from signal processing) used to study systems that change in discrete time steps — like the Laplace transform, but for discrete (step-by-step) systems instead of continuous ones.

$$X(z) = \sum x[n] \cdot z^{-n} \quad (n = 0 \text{ to } \infty)$$

Converts a discrete sequence $x[n]$ into frequency-domain form

Why Distributed Systems Need It

Distributed systems are discrete-event systems: messages arrive at set times, clocks tick in steps, retries happen after timeouts, queues change step-by-step, and agreement (consensus) happens in rounds → all of this can be modeled with difference equations → which is perfect for Z-transform analysis.

Main Uses

1. Network Delay & Congestion Analysis

$$q[n+1] = q[n] + a[n] - d[n]$$

q = queue length, a = arriving packets, d = departing packets

Used for: TCP traffic-jam control, making router buffers work better, and shaping traffic.

2. Distributed Control Systems

- Used in distributed databases & load balancing (e.g., cloud auto-scaling).
- Z-transform shows: whether scaling goes up and down unevenly, how fast it settles, and whether it stays stable when there is delay.

3. Clock Synchronization

Used in protocols like NTP, PTP, and sensor networks — which use repeating (recursive) clock-correction methods.

4. Reliability & Failure Recovery

Retry methods are modeled as discrete-time repeating (recursive) systems.

6. Cloud Computing

Delivery of computing services (servers, storage, databases, networking, software) over the internet — no need to own physical hardware.

NIST Definition

"A model for enabling convenient, on-demand network access to a shared pool of configurable computing resources."

5 Characteristics

Characteristic	Meaning
On-Demand Self-Service	Get resources anytime, without needing a person to help
Broad Network Access	Available over the internet from any device
Resource Pooling	Shared resources serve multiple users
Rapid Elasticity	Scale up/down quickly
Measured Service	Pay only for what you use

Deployment Models

Model	Owner	Key Trait
Public Cloud	Third-party provider (AWS, GCP, Azure)	Cheap, scalable, shared — less control/security
Private Cloud	Single organization	High control & security, but expensive
Hybrid Cloud	Mix of public + private	Flexible and cost-saving, but harder to manage
Community Cloud	Shared by orgs with similar needs	e.g., universities, government departments

Service Models

Model	Provides	Examples	Benefit
IaaS	Virtual machines, storage, networks	AWS EC2, Azure VMs	Full infrastructure control
PaaS	Platform to build/deploy apps	Google App Engine, Azure App Services	Faster development
SaaS	Ready-to-use software	Google Workspace, Microsoft 365	No installation, auto-updates

Major Public Cloud Platforms

Platform	Key Services	Strength
AWS	EC2, S3, RDS, Lambda	Largest service range, global infrastructure
Google Cloud (GCP)	Compute Engine, BigQuery, Kubernetes Engine	Strong AI/analytics + containers

Microsoft Azure	Virtual Machines, SQL DB, Active Directory	Best Microsoft product integration
-----------------	--	------------------------------------

Resource Management Functions

Function	Description
Resource Allocation	Assigning CPU, memory, storage
Load Balancing	Distributing workloads evenly
Virtualization	Creating virtual resources
Auto Scaling	Automatically increasing/decreasing resources
Monitoring	Tracking system performance

Security

Risks	Measures
Data breaches	Encryption
Unauthorized access	Authentication
Malware attacks	Firewalls
Data loss	Backup & Recovery
Insider threats	Access Control

★ Shared Responsibility Model

- The cloud provider secures the infrastructure
- The customer secures their own applications & data

7. Algorithms for Systems of Linear Equations

A system of linear equations is a set of equations that share the same variables. Important in engineering, machine learning, graphics, and scientific computing.

$$AX = B$$

A = coefficient matrix, X = unknowns, B = constants

Gaussian Elimination

- Step 1: Select the pivot element
- Step 2: Remove (zero out) the entries below the pivot
- Step 3: Repeat until an upper-triangular matrix is formed
- Step 4: Apply back substitution

Advantages	Disadvantages
Accurate for small systems	Costs a lot of computing power for large matrices
Deterministic (exact) solution	Scales poorly when run step-by-step (sequentially)

LU Decomposition

$$A = L \times U$$

L = lower triangular matrix, U = upper triangular matrix

- Benefit: efficient when solving multiple systems that share the same coefficient matrix A.
- Applications: scientific computing, numerical simulations, optimization problems.

Parallel Gaussian Elimination

What's Parallelized	Benefits	Challenges
Row operations, matrix partitioning, elimination	Less time to run, better hardware use, suits large matrices	Keeping processors in sync, extra communication cost, load balancing

Distributed Memory Systems: matrix blocks are distributed among nodes; nodes communicate via message passing (MPI — Message Passing Interface).

8. RFID (Radio Frequency Identification) — IoT

A wireless technology for automatically identifying and tracking objects using radio waves. A core building block of IoT.

Components

Component	Role
RFID Tag	Stores object info (Passive / Active / Semi-passive types)
RFID Reader	Reads data from tags
Antenna	Sends radio signals between the reader and the tag
Database/System	Stores & processes the collected data

Working Principle (4 Steps)

- 1. Reader sends a radio signal
- 2. RFID tag receives the signal
- 3. Tag sends its stored data back
- 4. Reader receives the data and forwards it to the database

Applications

Sector	Examples
Supply Chain	Inventory tracking, shipment monitoring, warehouse automation
Healthcare	Patient tracking, medicine ID, hospital asset management
Smart Transportation	Toll collection, vehicle tracking, smart parking
Retail	Automated billing, product tracking, theft prevention
Smart Agriculture	Livestock tracking, crop monitoring, equipment management

Advantages vs Challenges

Advantages	Challenges
Fast identification	Security concerns
Contactless communication	Privacy issues
Real-time tracking	High cost to set up
Increased automation	Signal interference
Reduced human effort	Limited reading range

9. Strassen's Matrix Multiplication

A divide-and-conquer algorithm that multiplies matrices FASTER than the usual method by trading multiplications for (cheaper) additions.

	Traditional	Strassen's
Multiplications (2x2)	8	7
Complexity	$O(n^3)$	$O(n^{2.81})$

■ Memory tip: 8 → 7: Strassen trades ONE multiplication for extra additions/subtractions — and additions are cheap!

The 7 Products (M1–M7)

$$M1 = (A11 + A22)(B11 + B22)$$

$$M2 = (A21 + A22) B11$$

$$M3 = A11 (B12 - B22)$$

$$M4 = A22 (B21 - B11)$$

$$M5 = (A11 + A12) B22$$

$$M6 = (A21 - A11)(B11 + B12)$$

$$M7 = (A12 - A22)(B21 + B22)$$

Result Sub-matrices

$$C11 = M1 + M4 - M5 + M7$$

$$C12 = M3 + M5$$

$$C21 = M2 + M4$$

$$C22 = M1 - M2 + M3 + M6$$

Pros & Cons

Advantages	Disadvantages
Faster for large matrices	Complex to implement
Suitable for parallel processing	Uses more memory
	Less efficient for very small matrices

Parallel Execution Model

Since M1–M7 are all independent, they can run in parallel:

Model	Approach
OpenMP (shared memory)	#pragma omp parallel sections — each thread computes one M_i (Thread 1→M1, ... Thread 7→M7)
MPI (distributed memory)	Master process sends A/B partitions to worker processes; each worker computes one M_i and sends results back

After all M1–M7 are computed, the master/root combines them into the result matrix C.

10. Iterative Methods for Linear Systems

Instead of solving exactly (like Gaussian Elimination), iterative methods start with a guess and keep improving it — great for large, sparse matrices.

Jacobi Method

Uses OLD values from the previous round to compute ALL new values at once.

Advantages	Disadvantages
Simple to implement	Takes longer to settle on an answer (slow convergence)
Easy to parallelize — great for distributed systems	Requires a diagonally dominant matrix

Gauss-Seidel Method

Uses NEWLY computed values right away (within the same round) — unlike Jacobi.

Advantages	Disadvantages
Settles on an answer faster than Jacobi	Harder to parallelize
Better accuracy	Each step depends on the one before it

Side-by-Side Comparison

Aspect	Jacobi	Gauss-Seidel
Uses values from	Previous round only	Current round (as soon as available)
Convergence speed	Slower	Faster
Parallelization	Easy (independent updates)	Difficult (depends on updated values)
Best fit	GPU / HPC / distributed systems	Single-processor / sequential systems

Convergence

$$||X^{k+1} - X^k|| < \epsilon$$

ϵ = small tolerance value; stop when the change is smaller than ϵ

Conditions: the matrix must be diagonally dominant, and the spectral radius must be less than 1.

Parallelizing Gauss-Seidel

Since it's naturally sequential, solutions include: block decomposition, domain decomposition, and hybrid methods.

11. OpenMP Quick Reference

OpenMP is a shared-memory API that lets you parallelize C/C++ code using simple `#pragma` directives.

Key Directives

Directive	Purpose
<code>#pragma omp parallel</code>	Creates a team of threads that run the block together
<code>#pragma omp parallel for</code>	Splits a for-loop's iterations across threads
<code>private(var)</code>	Each thread gets its own private copy of a variable
<code>reduction(+:sum)</code>	Safely combines a variable (e.g. sum) across all threads
<code>omp_get_thread_num()</code>	Returns the calling thread's ID number

Example: Parallel Sum with Reduction

```
#include <stdio.h>
#include <omp.h>
int main() {
    int i, sum = 0;
    int arr[10] = {1,2,3,4,5,6,7,8,9,10};
    #pragma omp parallel for reduction(+:sum)
    for (i = 0; i < 10; i++) {
        sum = sum + arr[i];
    }
    printf("Total Sum = %d\n", sum); // Output: 55
    return 0;
}
```

■ **Memory tip:** `reduction(+:sum)` avoids race conditions — each thread keeps its own private sum, then OpenMP adds them all together at the end.

12. Applications in Parallel & Distributed Systems

Domain	Examples
Scientific Computing	Weather prediction, fluid dynamics, molecular simulations
Machine Learning	Optimization algorithms, neural network training
Big Data Analytics	Distributed processing frameworks, parallel matrix operations
Computer Graphics	Image processing, 3D rendering, simulation systems
IoT & Smart Systems	RFID tracking, sensor data processing, distributed monitoring